

Data Preprocessing and Data Quality Report

Submitted By

Name: Ubaidullah Khan

Program: BS Artificial Intelligence (BSAI)

Course: Data Preprocessing / Machine Learning Lab

Instructor: MR.Kiramat

Submission Date: 27-July-2026

Data Preprocessing Report Using Kaggle Datasets

Abstract

The objective of this lab assignment is to perform complete data preprocessing on two Kaggle datasets by applying different data cleaning and feature engineering techniques. The preprocessing workflow includes identifying data quality issues, handling missing values, encoding categorical variables, scaling numerical features, creating new features, splitting data into training and testing sets, detecting data leakage, and building reusable preprocessing pipelines using Scikit-Learn.

The two datasets used are:

1. Titanic - Machine Learning from Disaster
 2. House Prices - Advanced Regression Techniques
-

Objectives

The objectives of this lab are:

- Identify common data quality issues.
- Handle missing values using different techniques.
- Remove duplicate and invalid records.

- Encode categorical variables.
 - Scale numerical variables.
 - Perform feature engineering.
 - Split the dataset properly.
 - Prevent data leakage.
 - Build reusable preprocessing pipelines.
 - Compare different preprocessing techniques.
-

Dataset 1: Titanic Dataset

Dataset Description

The Titanic dataset contains passenger information collected from the Titanic disaster. The objective is to predict whether a passenger survived based on several features.

Main Features

- PassengerId
 - Pclass
 - Name
 - Sex
 - Age
 - SibSp
 - Parch
 - Ticket
 - Fare
 - Cabin
 - Embarked
 - Survived (Target Variable)
-

Dataset 2: House Prices Dataset

Dataset Description

The House Prices dataset contains residential house information with the goal of predicting the selling price of houses.

Main Features

- LotArea
 - OverallQual
 - OverallCond
 - YearBuilt
 - YearRemodAdd
 - GarageCars
 - GarageArea
 - Neighborhood
 - SaleCondition
 - SalePrice (Target Variable)
-

Data Quality Assessment

Several data quality problems were identified before preprocessing.

1. Missing Values

Missing values were detected using:

```
df.isnull().sum()
```

Titanic

- Age
- Cabin
- Embarked

House Prices

Many columns contained missing values including:

- GarageType
 - GarageFinish
 - GarageQual
 - LotFrontage
 - FireplaceQu
 - PoolQC
 - Alley
-

2. Duplicate Records

Duplicate records were identified using:

```
df.duplicated().sum()
```

Duplicates were removed using:

```
df.drop_duplicates()
```

3. Invalid Values

Examples include:

- Negative Age
- Negative Fare
- Impossible SalePrice values

Such records were identified and corrected or removed.

4. Inconsistent Categories

Examples:

Male
male
MALE

These values were converted into a consistent format using string cleaning functions.

5. Incorrect Data Types

Some variables stored as object type were converted into numeric or datetime format wherever required.

6. Outliers

Outliers were detected using:

- Boxplots

- IQR Method
- Z-score

Examples:

- Extremely high Fare values
 - Very large house prices
 - Large LotArea values
-

7. Noisy Data

Noise was reduced by:

- Correcting inconsistent values
 - Removing impossible values
 - Rounding certain numerical features
 - Applying smoothing where necessary
-

Missing Value Treatment

Different techniques were compared.

Row Deletion

Rows containing missing values were removed when only a few records were affected.

Advantages:

- Simple
- Fast

Disadvantages:

- Loss of information
-

Column Deletion

Columns with excessive missing values (e.g., Cabin) were removed.

Advantages:

- Easy to implement

Disadvantages:

- Useful information may be lost.
-

Mean Imputation

Applied on numerical variables with approximately normal distribution.

Example:

Age

Median Imputation

Used when numerical features contained outliers.

Example:

LotArea

Mode Imputation

Used for categorical variables.

Example:

Embarked

GarageType

Advanced Imputation

KNN Imputer and Iterative Imputer were discussed as advanced methods.

Advantages:

- Better estimation
- Uses neighbouring observations
- Preserves relationships between variables

Categorical Data Encoding

Several encoding techniques were applied.

Label Encoding

Used for binary variables.

Example:

Sex

Male → 1

Female → 0

Ordinal Encoding

Applied to ordered categories such as quality ratings.

Poor < Fair < Good < Excellent

One-Hot Encoding

Applied to variables including:

Embarked

Neighborhood

SaleCondition

This method avoids introducing false ordering.

High Cardinality Variables

Columns with many unique values were handled using:

- Frequency Encoding
 - Target-aware techniques (discussion only)
-

Rare Category Handling

Rare categories with very small frequencies were grouped into "Other" to reduce overfitting.

Numerical Data Processing

Three scaling methods were compared.

Standardization

Transforms data to zero mean and unit variance.

Suitable for:

- Logistic Regression
 - SVM
 - Neural Networks
-

Min-Max Scaling

Scales values between 0 and 1.

Useful for:

- Deep Learning
 - Distance-based algorithms
-

Robust Scaling

Uses median and IQR.

Preferred when outliers exist.

Log Transformation

Applied to highly skewed variables such as:

- Fare
- SalePrice

This reduced skewness.

Outlier Treatment

Outliers were handled using:

- IQR Method
 - Z-score
 - Log Transformation
-

Feature Engineering

Several new features were created.

Titanic

FamilySize

= SibSp + Parch + 1

IsAlone

Derived from FamilySize

FarePerPerson

Fare $\div$ FamilySize

NameLength

Length of passenger name

AgeGroup

Created using age binning.

House Prices

HouseAge

= YrSold – YearBuilt

RemodelAge

= YrSold – YearRemodAdd

GarageRatio

GarageArea $\div$ GarageCars

OverallScore

OverallQual $\times$ OverallCond

Data Splitting

The dataset was divided into:

- Training Set
- Validation Set
- Testing Set

Typical split:

- Training = 70%
- Validation = 15%
- Testing = 15%

Different methods studied:

- Random Split
 - Stratified Split
 - Time-Based Split
-

Data Leakage

Different leakage situations were analysed.

Target Leakage

Using SalePrice while creating predictor variables.

Train-Test Contamination

Scaling the entire dataset before splitting.

Correct approach:

Split first, then preprocess.

Preprocessing Before Splitting

Incorrect:

Fit the scaler on complete data.

Correct:

Fit only on training data.

Future Information Leakage

Using future information while training models should always be avoided.

Scikit-Learn Pipeline

A reusable preprocessing pipeline was created using:

- Pipeline
- ColumnTransformer
- SimpleImputer
- StandardScaler
- OneHotEncoder

Benefits:

- Cleaner code
 - Reusability
 - Reduced human error
 - Prevention of data leakage
 - Easy deployment
-

Comparison of Missing Value Methods

Method	Advantages	Disadvantages
Row Deletion	Simple	Data loss
Column Deletion	Easy	Information loss
Mean	Fast	Sensitive to outliers
Median	Robust	Ignores relationships
Mode	Good for categories	Less suitable for numerical data
KNN Imputer	Accurate	Computationally expensive
Iterative Imputer	High accuracy	Slower

Practical Tasks Completed

- ✓ Data quality assessment
- ✓ Missing value handling
- ✓ Duplicate removal
- ✓ Invalid value correction
- ✓ Data type correction
- ✓ Outlier detection
- ✓ Noise reduction
- ✓ Label Encoding
- ✓ Ordinal Encoding
- ✓ One-Hot Encoding
- ✓ Handling high-cardinality variables
- ✓ Rare category handling
- ✓ Standardization
- ✓ Min-Max Scaling
- ✓ Robust Scaling

- ✓ Log Transformation
 - ✓ Feature Engineering
 - ✓ Data Splitting
 - ✓ Data Leakage Detection
 - ✓ Scikit-Learn Pipeline
-

Results

The preprocessing significantly improved the quality of both datasets. Missing values were successfully handled using multiple techniques. Outliers were reduced through statistical methods, categorical variables were properly encoded, numerical features were scaled, and new informative features were created. Proper train-test splitting and preprocessing pipelines ensured that data leakage was avoided and the datasets became suitable for machine learning models.

Conclusion

This lab demonstrated the complete preprocessing workflow required before building machine learning models. Two Kaggle datasets were cleaned using multiple preprocessing techniques. Various missing value treatments, encoding methods, scaling approaches, feature engineering techniques, and leakage prevention strategies were successfully implemented. Finally, reusable Scikit-Learn pipelines were created to automate preprocessing while maintaining reproducibility and preventing data leakage. This preprocessing pipeline prepares high-quality datasets for accurate and reliable machine learning model development.